

LLM PROMPT Chat — Math Flashcards Lab

Task 1: Fix String Concatenation Calculation Bug

Problem

The `compute_expected_answer()` function incorrectly concatenates the two operands instead of performing the mathematical operation.

For example:

$7 + 5$

was incorrectly treated as:

75

instead of:

12

LLM Prompt Used

I am working on a Pygame Math Flashcards lab.

Task 1 is to fix a deliberate bug in `game/game_engine.py`.

Current bug:

The `compute_expected_answer()` method incorrectly returns:

```
int(f"{self.num_a}{self.num_b}")
```

This concatenates the two operands as strings. For example:

7 + 5 incorrectly produces 75 instead of 12.

Required change:

Refactor `compute_expected_answer()` so that it checks `self.operator` and performs the actual mathematical operation:

- "+" → `self.num_a + self.num_b`

- "-" → `self.num_a - self.num_b`

- "*" → `self.num_a * self.num_b`

Requirements:

1. Make the smallest possible change needed to fix Task 1.
2. Do not modify unrelated functionality.
3. Preserve the existing class structure and variable names.
4. Handle the three operators explicitly.
5. Explain exactly what was changed and why.

Change Made

The `compute_expected_answer()` method was changed to check `self.operator` and perform the corresponding arithmetic operation.

Expected Behavior

7 + 5 → 12

9 - 4 → 5

6 * 3 → 18

Testing

The original game was tested before making the change to demonstrate the calculation bug.

After the change, the game was tested again with addition, subtraction, and multiplication.

LLM Used

ChatGPT / GitHub Copilot

Chat History

I am working on a Pygame Math Flashcards lab. Task 1 has already been completed, so do not undo or modify the existing arithmetic calculation fix.

Now implement ONLY Task 2 in game/game_engine.py:

TASK 2 — Per-question 10-second timer bar

Requirements:

1. Add a 10-second countdown timer for each flashcard question.
2. Display a visible horizontal timer/progress bar below the card container in the existing render() method.
3. The timer should start at 10 seconds whenever a new flashcard is generated.

4. Update the remaining time inside the existing update() method using Pygame's timing mechanism/delta time so that the countdown is based on real elapsed time rather than frame count.

5. The timer bar should visually decrease as the remaining time decreases.

6. When the timer reaches zero BEFORE the player submits an answer:

- Count the question as an incorrect attempt.
- Display exactly or clearly as "TIME'S UP!".
- Move to the next flashcard.
- Reset the timer for the new question.
- Do not allow the same timeout to be counted multiple times.

7. Preserve the existing behavior for:

- Correct answers
- Incorrect answers
- Empty input
- Score
- Attempts
- TextBox
- Submit button
- Existing card generation
- Task 1 arithmetic calculation fix

8. Do not implement Task 3 streak multipliers yet.

9. Do not implement Task 4 division yet.

10. Keep the existing object-oriented structure, variable names, rendering style, and coding style as much as possible.

11. Make the smallest reasonable changes necessary. Do not rewrite the entire game or unrelated functions.

12. If the existing code already has a timing mechanism, reuse it rather than creating a conflicting timing system.

13. Make sure the timer resets whenever a new card is created, including after a timeout and after a normal submitted answer that moves to the next card.

14. Make sure the timer does not continue running after the game has ended, if the project has a game-over state.

After making the changes:

- Explain which variables/methods were added or modified.
- Explain how the countdown works.
- Explain how timeout is counted as an incorrect attempt.
- Show the final relevant code sections.
- Identify anything I should manually test.

Task 3: consecutive correct streak multipliers.

I am working on a Pygame Math Flashcards lab.

Task 1 (arithmetic calculation fix) and Task 2 (10-second per-question timer) have already been implemented and tested.

Now implement ONLY Task 3: consecutive correct streak multipliers.

Requirements:

1. Add a streak counter to the game engine that tracks the number of consecutive correct answers.
2. Every time the player submits a correct answer:
 - Increase the consecutive streak by 1.
 - Award points based on the current streak multiplier.

3. Implement these multiplier rules:

- Streak 0–2: 1x multiplier
- Streak 3–4: 2x multiplier
- Streak 5 or more: 3x multiplier

Therefore:

- 1st correct answer → +1 point
- 2nd consecutive correct → +1 point
- 3rd consecutive correct → +2 points
- 4th consecutive correct → +2 points
- 5th consecutive correct → +3 points
- 6th consecutive correct → +3 points
- Continue using 3x for streaks of 5 or more.

4. Reset the streak counter to 0 whenever:

- The player submits an incorrect answer.
- The timer reaches zero / TIME'S UP.

5. After a reset, the next correct answer should start a new streak at 1x.

6. The score should increase by the multiplier amount rather than always increasing by exactly 1.

7. If appropriate, display the current streak and/or multiplier in the existing game UI, but do not redesign the UI unnecessarily.

8. Preserve all existing functionality:

- Task 1 arithmetic calculation fix
- Task 2 10-second timer
- Timer progress bar
- TIME'S UP behavior

- Incorrect-answer behavior
- Empty-input behavior
- TextBox behavior
- Submit button
- Existing card generation

9. Do NOT implement Task 4 division yet.

10. Do not rewrite the entire game. Make the smallest reasonable changes to the existing object-oriented code.

11. Make sure a timeout resets the streak exactly once and does not accidentally award points.

12. Make sure an incorrect answer resets the streak but preserves the existing attempt/feedback behavior.

13. Avoid changing unrelated code.

After making the changes:

- Explain which variables and methods were added or modified.
- Explain exactly how the multiplier is calculated.
- Show the relevant final code.
- Explain how incorrect answers and timeouts reset the streak.
- Give me a short manual testing checklist with expected scores for streaks of 1, 2, 3, 4, and 5.

Task 4: integer division support.

I am working on a Pygame Math Flashcards lab.

Task 1 (arithmetic calculation fix), Task 2 (10-second timer), and Task 3 (consecutive correct streak multipliers) have already been implemented and tested successfully.

Now implement ONLY Task 4: integer division support.

Requirements:

1. Add the "/" operator to `game_engine.generate_new_card()`.
2. Division questions must ALWAYS produce a positive whole-number answer with no remainder.
3. Do NOT generate arbitrary dividend and divisor values and then use normal division, because that could produce decimal results.
4. Instead, generate a divisor and a quotient first, then calculate the dividend:

$$\text{dividend} = \text{divisor} * \text{quotient}$$

This guarantees:

$$\text{dividend} / \text{divisor} = \text{quotient}$$

5. Example valid questions:

$$20 / 5 = 4$$

$$18 / 3 = 6$$

$$35 / 7 = 5$$

6. Do not generate division by zero.
7. Make sure the generated operands fit the existing number range/style of the game.
8. Update `compute_expected_answer()` so that when `self.operator == "/"`,
it returns the correct integer division result.
9. Since the generated division problems are guaranteed to divide evenly, the answer must be an integer with no remainder.

10. Preserve all existing functionality:

- Addition
- Subtraction
- Multiplication
- Task 1 calculation fix
- Task 2 timer and TIME'S UP behavior
- Task 3 streak multipliers
- Attempts
- Score
- TextBox
- Existing UI

11. Do not change unrelated code.

12. Do not rewrite the entire game.

13. Do not introduce floating-point answers for division.

14. After implementing the change, explain:

- How division cards are generated.
- How division by zero is prevented.
- How `compute_expected_answer()` handles `"/"`.
- Which existing functions were modified.
- What manual tests I should perform.

Also check that the existing `+`, `-`, and `*` operators continue to work.